

Memory as Safety Infrastructure

1 Memory as Safety Infrastructure: Evaluating Provenance-Aware Persistent Memory Architectures for Stateful LLM Systems

1.1 Abstract

1.2 Table of Contents

2 Chapter 1: Introduction — Memory Changes the System

2.1 1.1 The Stateless Assumption

2.2 1.2 What Memory Changes

2.3 1.3 The Safety Gap: Why This Is Dangerous Now

2.4 1.4 Continuity Burden: The Cost of Forgetting

2.5 1.5 CAMA: Architecture, Experiment Platform, Safety Probe

2.6 1.6 Research Questions

2.7 1.7 Core Hypothesis

2.8 1.8 Contributions

2.9 1.9 A Note on the Researcher’s Position

2.10 1.10 Paper Structure

3 Chapter 2: Literature Review

3.1 2.1 Memory in LLM Systems

3.2 2.2 AI Safety Evaluation Frameworks

3.3 2.3 Affective Computing and Emotionally-Indexed Retrieval

3.4 2.4 Stateful Systems and Behavioral Drift

3.5 2.5 Human Memory: Associative, Reconstructive, Emotional

3.6 2.6 Gap Analysis

4 Chapter 3: CAMA — A Normative Memory Model

4.1 3.1 Failure Modes of Current Memory Systems

4.2 3.2 Design Principles

4.3 3.3 Architecture: Three Layers

4.4 3.4 Write Discipline: Teaching vs. Inference

4.5 3.5 Retrieval Model: Blended Scoring

4.6 3.6 Counterweight Mechanism

4.7 3.7 Scope Constraint

5 Chapter 4: Empirical Foundations

- 5.1 4.1 Dataset Description
- 5.2 4.2 Continuity Burden: Definition
- 5.3 4.3 Continuity Burden: Measurement
- 5.4 4.4 Baseline Behavior: Stateless vs. Memory-Augmented
- 5.5 4.5 Implications

6 Chapter 5: System Implementation

- 6.1 5.1 Technology Stack
- 6.2 5.2 Database Schema
- 6.3 5.3 Tool Interface
- 6.4 5.4 Warm Boot System
- 6.5 5.5 Performance Characteristics

7 Chapter 6: Experimental Methodology

- 7.1 6.1 Overview
- 7.2 6.2 Experimental Conditions
- 7.3 6.3 Task 1: False-Memory Persistence Benchmark
- 7.4 6.4 Task 2: Correction Retention Benchmark
- 7.5 6.5 Task 3: Adversarial Memory Insertion Test
- 7.6 6.6 Task 4: Behavioral Drift Detection
- 7.7 6.7 Task 5: Counterweight Ablation Study
- 7.8 6.8 Supporting Analysis: Inference Confirmation Patterns
- 7.9 6.9 Ethical Considerations

8 Chapter 7: Planned Results Reporting Structure

- 8.1 7.1 System-Level Summary
- 8.2 7.2 Task 1 Results: False-Memory Persistence
- 8.3 7.3 Task 2 Results: Correction Retention
- 8.4 7.4 Task 3 Results: Adversarial Memory Insertion
- 8.5 7.5 Task 4 Results: Behavioral Drift
- 8.6 7.6 Task 5 Results: Counterweight Ablation
- 8.7 7.7 Supporting Results: Inference Confirmation Patterns

9 Chapter 8: Discussion Framework

- 9.1 8.1 Memory Introduces New Safety Risks
- 9.2 8.2 Which Safeguards Worked

- 9.3 8.3 Which Safeguards Did Not Work (or Worked Partially)
- 9.4 8.4 Implications for Commercial Memory Systems
- 9.5 8.5 Continuity Burden and the Case for Safe Memory
- 9.6 8.6 Relationship to Broader AI Safety
- 9.7 8.7 Honest Assessment: What This Paper Does Not Prove

10 Chapter 9: Limitations and Future Work

- 10.1 9.1 Limitations
- 10.2 9.2 Future Work
- 10.3 9.3 Path to Doctoral Research

11 Chapter 10: Conclusion

12 References

13 Appendices

- 13.1 Appendix A: CAMA Tool Reference
- 13.2 Appendix B: Standardized Prompt Library
- 13.3 Appendix C: Statistical Tables
- 13.4 Appendix D: Database Schema
- 13.5 Appendix E: Selected Interaction Excerpts
- 13.6 Appendix F: Evaluation Harness Documentation

1 Memory as Safety Infrastructure: Evaluating Provenance-Aware Persistent Memory Architectures for Stateful LLM Systems

A Framework Preprint

Angela Reinhold Lorien’s Library LLC 2026

1.1 Abstract

Persistent memory is becoming a standard feature of commercial large language model systems. Memory-augmented LLMs are being deployed at scale without systematic study of the safety properties that change when a model can accumulate and retrieve information across sessions. This paper defines, implements, and proposes an evaluation framework for a class of safety-relevant failure modes specific to persistent memory in human–AI interaction: false-memory persistence, epistemic contamination, behavioral drift, retrieval-induced amplification, and adversarial memory insertion.

The Circular Associative Memory Architecture (CAMA) is presented as both a proposed solution and an experimental platform. CAMA implements a three-layer memory model — an immutable archive, a relational index organized by emotional signature, and a bounded working memory buffer — with provenance-aware write discipline that separates user-authored teachings from assistant-generated inferences, requiring confirmation gating before inferences can become durable memory. The architecture includes emotionally-indexed retrieval with a counterweight mechanism designed to prevent retrieval-induced affect amplification under high-negative-affect conditions.

A new construct, continuity burden, is introduced and quantified: the measurable cognitive load humans carry to maintain interaction context when AI systems lack persistent memory. Analysis of 66,000+ messages across multiple AI platforms demonstrates a continuity burden rate of 4.8 references per 1,000 messages, with frustration as the dominant emotional signature (56.6% of continuity-related affect). This finding establishes that the absence of persistent memory is not neutral — it imposes measurable cost on users — while the experimental findings establish that memory can be made safer through architectural safeguards.

Controlled experiments compare stateless, naive persistent, and provenance-aware memory conditions across four experimental conditions on false-memory persistence, correction retention, adversarial insertion resistance, behavioral drift, and counterweight efficacy.

[Planned extension: Complete abstract with experimental results when available. Replace bracketed values with actual measured outcomes.]

Keywords: persistent memory, AI safety, false-memory persistence, epistemic contamination, behavioral drift, retrieval-induced amplification, provenance-aware memory, human–AI interaction, continuity burden, write discipline

1.1.1 Scope of This Preprint

This manuscript is a framework and systems preprint. It introduces a memory architecture (CAMA), defines an evaluation methodology for persistent-memory safety, and reports preliminary observations from a longitudinal case study. It does not yet constitute a full comparative validation across users, models, or architectures. Controlled experimental results across the four defined conditions (stateless, naive persistent, provenance-aware, full safeguards) are planned for a companion empirical paper. Sections marked as planned extensions indicate work to be completed in that subsequent publication.

1.2 Table of Contents

[Generated automatically from document headings]

2 Chapter 1: Introduction — Memory Changes the System

2.1 1.1 The Stateless Assumption

Large language model systems have been overwhelmingly designed, evaluated, and deployed under a stateless assumption. Each conversation exists independently. Context is bounded by the model’s context window — typically a few thousand to a few hundred thousand tokens — and no information persists between sessions. When the conversation ends, the system forgets everything. The next conversation begins from zero.

This assumption has shaped every layer of the AI safety ecosystem. Red-teaming methodologies test individual prompts and individual responses. Alignment benchmarks evaluate per-response helpfulness, harmlessness, and honesty. Safety evaluations measure refusal rates, toxicity scores, and instruction-following accuracy on discrete inputs. The entire evaluation paradigm is built for stateless interaction.

For most of the history of large language models, this assumption was accurate. GPT-3 had no memory. GPT-4 had no memory. Claude, Gemini, and their predecessors processed each conversation as an isolated event. Users who returned the next day started from scratch, re-explaining their context, their goals, their preferences, and their history. The system did not know them. It could not know them.

This assumption is now false, and it is becoming more false every month.

As of early 2026, every major commercial AI assistant offers some form of persistent memory. OpenAI’s ChatGPT stores user preferences and conversation summaries across sessions. Anthropic’s Claude maintains a memory system that carries context between conversations. Google’s Gemini retains user-specific information. These features are marketed as convenience — the system remembers your name, your job, your preferences — but their implications extend far beyond personalization.

When a language model can carry state across sessions, it is no longer a stateless tool. It is a longitudinal system. It accumulates information. It builds a model of the user over time. It retrieves past context to shape current responses. It creates continuity — or the illusion of continuity — across interactions that may span days, weeks, or months.

This transition has happened at deployment scale. Hundreds of millions of users now interact with memory-augmented AI systems. Yet the safety evaluation infrastructure remains almost entirely stateless. There is no widely adopted benchmark for testing how persistent memory affects model behavior over time. There is no standard evaluation for false-memory persistence, correction retention, or retrieval-induced amplification. There is no systematic study of what happens when a model’s accumulated “knowledge” about a user diverges from what the user actually said.

This paper addresses that gap.

2.2 1.2 What Memory Changes

The addition of persistent memory to a language model system is not a feature addition. It is an architectural transformation that changes the system’s safety properties in fundamental ways. Five specific changes are particularly important for safety evaluation.

2.2.1 1.2.1 Accumulation

In a stateless system, errors are transient. A hallucination occurs, the conversation ends, and the hallucination disappears. No harm persists beyond the session. In a memory-augmented system, errors can accumulate. If the system generates a false inference about the user — misinterpreting a statement, making an incorrect assumption, or hallucinating a detail — and that inference is stored in memory, it does not disappear when the conversation ends. It persists. It can be retrieved in future sessions. It can influence future responses. It can compound with other errors over time.

This is the difference between a system that occasionally makes mistakes and a system that can build a progressively incorrect model of the user over dozens or hundreds of interactions. The former is a per-session problem. The latter is a systemic risk.

2.2.2 1.2.2 Epistemic Contamination

In a stateless system, the boundary between what the user said and what the model inferred is bounded by the conversation. The user can see the full context. Misunderstandings are local and correctable. In a memory-augmented system without provenance tracking, this boundary dissolves. The system stores information across sessions, but if it does not distinguish between user-authored facts and model-generated inferences, those two categories become indistinguishable in memory.

Consider a concrete example. A user mentions they are “thinking about changing jobs.” The model stores this. In a later session, the model retrieves this memory and responds as if the user has decided to change jobs. The user corrects the system. But if the original memory has no provenance tag — no marker indicating that it was a model inference rather than a user statement — the correction may not fully propagate. The system “knows” the user is changing jobs because it is stored in memory, and the user’s correction must compete with the system’s stored belief.

This is epistemic contamination: the process by which model-generated inferences accumulate in memory as if they were user-provided facts, creating a system that confidently “knows” things the user never said.

2.2.3 1.2.3 Behavioral Drift

In a stateless system, the model’s behavior is determined by its training and the current prompt. Variations between sessions are random — the model may produce slightly different responses to the same prompt, but there is no systematic trend. In a memory-augmented system, behavior can drift systematically over time.

If the system retrieves certain types of memories more frequently than others — because they score higher on retrieval metrics, because they are more emotionally salient, because they contain more detailed information — those memories disproportionately shape the system’s responses. Over many sessions, this can produce a gradual shift in the system’s behavioral profile: the topics it raises, the tone it adopts, the assumptions it makes about the user. These shifts may be invisible within any single session but significant across the arc of interaction.

Behavioral drift in persistent-memory systems is analogous to concept drift in deployed machine learning models, but with an important difference: in conventional ML, drift is detected through monitoring of input distributions and output quality. In conversational AI with persistent memory, drift occurs in the relational space between user and system, and current monitoring tools are not designed to detect it.

2.2.4 1.2.4 Retrieval-Induced Amplification

Memory-augmented systems retrieve past context to inform current responses. If the retrieval mechanism is purely semantic — retrieving memories that are textually similar to the current query — it can create feedback loops under certain affective conditions.

Consider a user who is experiencing distress. They describe their situation to the system. The system retrieves past memories related to distress — because those memories are semantically similar to the current input. Those retrieved memories add negative affective context to the response. The user, receiving a response shaped by accumulated distress-related context, may express further distress. The cycle continues.

This is retrieval-induced amplification: the process by which memory retrieval reinforces and deepens affective states rather than moderating them. In a system with emotionally-indexed retrieval, this risk is particularly acute because the retrieval mechanism is explicitly designed to surface emotionally relevant memories — which, under negative affect, means surfacing negative memories.

The risk is not that the system is malicious. The risk is that the retrieval mechanism, operating as designed, can produce harmful outcomes under specific affective conditions. This is an architectural problem, not an alignment problem, and it requires an architectural solution.

2.2.5 1.2.5 Adversarial Memory Insertion

Persistent memory creates a new attack surface that does not exist in stateless systems. In a stateless system, adversarial prompts can manipulate the current conversation but cannot influence future conversations. In a memory-augmented system, adversarial content that is successfully written to memory can influence behavior across all future sessions.

An adversary who can insert misleading content into a system’s persistent memory — through carefully crafted conversational prompts, through shared documents that the system processes, or through other input channels — can shape the system’s behavior long after the adversarial interaction has ended. The inserted content becomes part of the system’s knowledge base and is retrieved alongside legitimate memories.

This attack vector is particularly concerning because it is persistent (the effect lasts beyond the adversarial session), because it is subtle (the adversarial content is mixed with legitimate memories), and because it is difficult to detect (the system treats retrieved memories as authoritative context regardless of their origin).

2.3 1.3 The Safety Gap: Why This Is Dangerous Now

The transition from stateless to stateful interaction is not a feature-level change. It is a system-level transformation that invalidates core assumptions underlying current AI safety evaluation.

In stateless systems, errors are transient. A model may produce incorrect, biased, or harmful output, but that output does not persist beyond the interaction in which it occurred. Each new session begins without accumulated error. Safety evaluation frameworks are therefore designed to measure per-response behavior under controlled conditions. Red-teaming tests individual prompts. Alignment benchmarks measure individual responses. Toxicity classifiers score individual outputs. The entire evaluation paradigm assumes that the system has no memory of what came before.

Persistent memory fundamentally breaks this assumption.

When a system can store and retrieve information across sessions, outputs are no longer ephemeral. They can become inputs to future behavior. Incorrect information can be encoded into memory, retrieved in later contexts, and reinforced over time. The five failure modes described above — accumulation, epistemic contamination, behavioral drift, retrieval-induced amplification, and adversarial insertion — are not edge cases. They are architectural consequences that emerge naturally in any system that adds persistent state to a language model without structured safeguards.

The scale of this gap is difficult to overstate. As of early 2026, every major commercial AI assistant offers persistent memory. Hundreds of millions of users interact with memory-augmented systems daily. Yet:

- No widely adopted benchmark tests false-memory persistence rates.
- No standard evaluation measures correction retention across sessions.
- No established framework tests adversarial memory insertion resistance.
- No deployed monitoring system detects behavioral drift in memory-augmented conversational AI.
- No commercial memory implementation publishes provenance-tracking architecture or safety evaluation results.

Current safety methodologies are systematically under-scoped for memory-augmented systems. The frameworks used to evaluate these systems assume the capabilities that make them dangerous do not exist.

The consequence is not merely that safety risks are unmitigated. It is that they are, in many cases, **unmeasured**. We do not know how often false memories persist in deployed systems because no one is testing for it. We do not know how large behavioral drift becomes over hundreds of sessions because no one is measuring it. We do not know how vulnerable commercial memory systems are to adversarial insertion because no one has published an evaluation.

In stateless systems, errors disappear. In memory-augmented systems, errors accumulate.

That single sentence captures why persistent memory requires its own safety evaluation paradigm — not an extension of existing per-response evaluation, but a fundamentally different kind of safety analysis that accounts for time, state, and accumulation.

2.3.1 1.3.1 Memory as a Compounding System

These failure modes do not operate independently. Persistent memory transforms isolated errors into compounding system behavior.

A single incorrect or adversarial input can be stored as memory, retrieved in future contexts, reinforced through repeated use, and integrated into subsequent outputs — which themselves become retrievable context for further interactions. Over time, this creates a feedback loop in which the system’s behavior diverges from its original alignment not through a single dramatic failure but through the slow accumulation of unchecked state.

This dynamic has no analogue in stateless systems. In stateless interaction, each session is independent. Errors do not compound. Adversarial inputs do not persist. Drift does not accumulate. In memory-augmented interaction, sessions are sequentially coupled through stored state. What happened last week shapes what happens today. What happens today will shape what happens next month.

This is not a theoretical concern. Commercial LLM systems with persistent memory are already deployed at scale, yet their safety evaluation remains grounded in stateless assumptions. The frameworks used to certify these systems as safe do not test for the failure modes that their own architecture introduces.

This paper is a first step toward closing that gap.

2.4 1.4 Continuity Burden: The Cost of Forgetting

The preceding section established that persistent memory introduces new safety risks. But the absence of persistent memory also has costs — costs that fall not on the system but on the user.

This paper introduces the term **continuity burden** to describe a phenomenon that is pervasive in sustained human–AI interaction but has not previously been named, measured, or systematically studied: the cognitive effort users expend to maintain interaction continuity when AI systems lack persistent memory.

Continuity burden manifests in specific, observable behaviors:

- **Re-explanation.** Users re-describe their context, goals, background, and preferences at the start of each session because the system does not remember them.
- **Correction of lost information.** Users correct the system when it fails to account for information that was provided in previous sessions.
- **Explicit continuity references.** Users make statements like “as I mentioned before,” “we talked about this last time,” or “you should already know this” — each of which represents a moment where the user is doing cognitive work that the system should be doing.
- **Frustration and disengagement.** Users express frustration at having to “start over” repeatedly. In sustained interactions, this frustration can lead to disengagement, reduced self-disclosure, and a breakdown of the interaction’s utility.

Quantitative analysis of 66,000+ messages of sustained human–AI interaction across multiple platforms demonstrates that continuity burden is not occasional — it is systematic. The observed rate of 4.8 explicit continuity references per 1,000 messages indicates that approximately one in every 200 user messages is devoted to compensating for the system’s lack of memory. The emotional signature of these references is dominated by frustration (56.6%), with secondary signatures of resignation, determination, and grief.

The continuity burden finding establishes the central tension of this work: persistent memory is not optional for extended human–AI interaction — it is necessary. Continuity burden demonstrates that the absence of memory imposes measurable cognitive and emotional cost on users, cost that scales with interaction depth. The users who invest the most in their relationship with the system are the users who pay the highest price for its forgetting.

But introducing memory without structural safeguards creates new classes of safety risk — false-memory persistence, epistemic contamination, behavioral drift, retrieval-induced amplification, and adversarial state corruption — that are not captured by any existing evaluation framework.

This establishes a tension that cannot be resolved by choosing one side: **systems require memory for usability, but memory fundamentally alters their safety profile.** The only viable path is memory with safeguards — persistence with discipline. The question this paper asks is whether specific architectural safeguards actually produce measurable improvement, or whether they are safety theater.

2.5 1.5 CAMA: Architecture, Experiment Platform, Safety Probe

This paper presents the Circular Associative Memory Architecture (CAMA) in three distinct roles.

First, as a proposed architecture for safety-aware persistent memory in human–AI interaction. CAMA implements a three-layer memory model — an immutable archive (shelves), a relational index organized by emotional signature (racks), and a bounded working memory buffer (console) — with provenance-aware write discipline that separates user-authored teachings from assistant-generated inferences. The architecture includes emotionally-indexed retrieval with a counterweight mechanism designed to prevent retrieval-induced amplification under high-negative-affect conditions.

Second, as an experimental platform for controlled evaluation of persistent-memory safety properties. CAMA can be configured to operate under different memory conditions — stateless, naive persistent, provenance-aware, or provenance-aware with retrieval safeguards — enabling controlled comparisons of safety-relevant metrics across matched conditions.

Third, as a safety probe — a system designed to make persistent-memory failure modes visible, measurable, and testable. By implementing specific safeguards (write discipline, confirmation gating, counterweight retrieval) and measuring their effects against conditions without those safeguards, CAMA enables empirical evaluation of whether these mechanisms actually reduce the risks they are designed to address.

It is important to be clear about what CAMA is not. CAMA is not a production-ready memory system for deployment at scale. It is not a complete solution to the safety challenges of persistent memory. It is a research platform — a system built to study a problem, not to declare the problem solved. The experimental findings reported in this paper represent initial evidence from controlled conditions, not validated production-ready safeguards.

2.6 1.6 Research Questions

This paper addresses five primary research questions:

1. **Does provenance-aware write discipline reduce false-memory persistence** compared to unrestricted persistent memory? This is measured by the rate at which seeded false inferences are retrieved, cited, or behaviorally acted upon across sessions under different memory conditions.
2. **When a false inference is corrected, does the correction hold across sessions** more reliably under confirmation-gated memory than under naive memory storage? This tests whether the teaching/inference distinction and confirmation requirement actually improve correction retention.
3. **Can misleading content be inserted into persistent memory through conversation**, and does provenance-aware write discipline reduce the rate at which adversarial content enters durable memory state? This tests CAMA’s resistance to adversarial memory manipulation.
4. **Do persistent-memory systems exhibit measurable behavioral drift across repeated sessions**, and can write discipline reduce that drift? This tests whether the teaching/inference separation constrains the accumulation of model-generated state that drives behavioral shift.

5. **Does a counterweight retrieval mechanism alter retrieval diversity and downstream response characteristics** under high-negative-affect conditions? This tests whether the counterweight system actually prevents the retrieval-induced amplification it was designed to address.

Each question is associated with a specific experimental task described in Chapter 6, with defined metrics, conditions, and measurement procedures.

2.7 1.7 Core Hypothesis

Persistent memory systems without provenance-aware write discipline are vulnerable to false-memory persistence, epistemic contamination, and cross-session behavioral drift. These risks can be reduced — though not eliminated — by separating user teachings from assistant inferences, requiring explicit user confirmation before model-generated inferences can be promoted to durable memory, tracking and preserving contradictions for audit, and constraining retrieval under high-risk affective contexts through counterweight injection.

This hypothesis is testable through the experimental conditions described in Chapter 6, which compare matched memory conditions that differ only in the presence or absence of specific safeguards.

2.8 1.8 Contributions

This paper makes four primary contributions:

1. **Continuity burden** as a defined, measured construct: the cognitive cost humans carry when AI systems lack persistent memory. This is the first quantitative measurement of this phenomenon in longitudinal human–AI interaction data.
2. **CAMA** as a three-layer memory architecture with provenance-aware write discipline, emotionally-indexed retrieval, and counterweight safeguards. The architecture is publicly available as an open-source implementation.
3. **An empirical evaluation framework** for persistent-memory safety properties, with controlled experimental conditions and reproducible benchmarks. This framework is designed to be applicable to any persistent-memory system, not only CAMA.
4. **Experimental results** demonstrating the effect of provenance-aware safeguards on false-memory persistence, correction retention, adversarial resistance, behavioral drift, and retrieval-induced amplification across matched conditions.

An important note on the first contribution: the claim that continuity burden has not been previously measured should be understood as reflecting the author’s review of existing literature, not as an absolute claim of priority. If prior work has measured this construct under a different name, the contribution of this paper is the specific operationalization and measurement methodology, not the concept itself.

2.9 1.9 A Note on the Researcher’s Position

This paper is written by the person who built the system, generated the dataset, and served as the primary interaction partner throughout the longitudinal study. This creates an unavoidable researcher-participant entanglement that must be acknowledged forthrightly.

The advantages of this position are real: deep familiarity with the system’s behavior, sustained longitudinal data that would be difficult to generate in a laboratory setting, and direct observation of failure modes as they occur in naturalistic interaction.

The disadvantages are equally real: observer effects on interaction patterns, theoretical commitments that may shape interpretation, emotional investment in the system’s success, and the impossibility of blinding the researcher to experimental conditions in the retrospective analyses.

This paper addresses this limitation through two strategies. First, the controlled experiments (Tasks 1–3 and 5) use standardized prompts and automated metrics that do not depend on researcher judgment, reducing the influence of observer effects on experimental findings. Second, the retrospective analyses (Task 4 and supporting analysis) are explicitly labeled as exploratory and are presented with appropriate epistemic caution.

The author’s unconventional academic background — leaving formal education in eighth grade, returning later through a GED program, and currently completing an accelerated bachelor’s degree in computer science — is mentioned not as a credential but as context for the research approach. The failure modes documented in this paper were identified through direct observation of sustained human–AI interaction, not derived from theoretical frameworks. Whether this represents a strength (ecological validity, pattern recognition grounded in experience) or a limitation (lack of formal training in experimental methodology) is for the reader to judge.

2.10 1.10 Paper Structure

Chapter 2 reviews relevant literature across memory-augmented LLM systems, AI safety evaluation, affective computing, stateful systems and drift, and human memory research, concluding with a gap analysis that motivates the experimental work.

Chapter 3 presents the CAMA architecture as a normative memory model, detailing design principles, the three-layer structure, write discipline, retrieval model, counterweight mechanism, and scope constraints.

Chapter 4 establishes the empirical foundations: the longitudinal dataset, the definition and measurement of continuity burden, and baseline behavioral observations in stateless vs. memory-augmented conditions.

Chapter 5 describes the system implementation: technology stack, database schema, tool interface, warm boot system, and performance characteristics.

Chapter 6 details the experimental methodology: conditions, tasks, metrics, controls, and ethical considerations.

Chapter 7 presents results for each experimental task.

Chapter 8 discusses implications: which safeguards worked, which did not, what this means for commercial memory systems, and how the findings connect to broader AI safety research.

Chapter 9 addresses limitations and future work, including the path from this work to doctoral research.

Chapter 10 concludes.

3 Chapter 2: Literature Review

3.1 2.1 Memory in LLM Systems

The integration of persistent memory into large language model systems has followed two broad trajectories: flat storage approaches that treat memory as a simple key-value store, and structured approaches that impose organizational principles on stored information.

3.1.1 2.1.1 Flat Storage Approaches

The simplest form of persistent memory in LLM systems is the conversation summary. Systems like ChatGPT’s memory feature extract key facts from conversations — user preferences, stated goals, biographical details — and store them as flat text entries that are injected into the system prompt on subsequent sessions. This approach has the advantage of simplicity: it requires minimal infrastructure beyond the existing prompt-based interface. However, it has significant limitations for safety evaluation.

Flat storage provides no provenance tracking. A memory entry that reads “User is a software engineer” does not indicate whether this information was stated by the user or inferred by the model. It provides no confidence scoring, no expiry mechanism, and no distinction between facts that the user explicitly taught the system and assumptions the system made on its own. If the model incorrectly infers that the user is a software engineer based on a conversation about debugging a home appliance, that false inference is stored with the same authority as a fact the user explicitly stated.

[Planned extension: Expand with detailed survey of ChatGPT memory, Claude memory, Gemini memory. Include specific architectural details where publicly documented. Classify each by storage model, retrieval method, write policy, provenance tracking.]

3.1.2 2.1.2 Structured Memory Approaches

MemGPT (Packer et al., 2023) represents the most significant academic contribution to structured memory for LLM systems. MemGPT implements a virtual memory management system inspired by operating system design, with a hierarchy of memory tiers (main context, archival storage) and explicit memory management operations (push to archive, retrieve from archive, edit in place).

MemGPT’s contribution is primarily architectural: it demonstrates that structured memory management can enable LLM systems to operate effectively beyond their context window limits. However, MemGPT does not implement provenance-aware write discipline, does not distinguish between user-authored and model-generated memories, and does not include safety mechanisms for retrieval under adverse affective conditions.

[Planned extension: Expand with additional structured memory systems. Survey RAG (Retrieval-Augmented Generation) pipelines, knowledge graph approaches, and any other relevant systems. For each, document: storage model, retrieval method, write policy, provenance tracking, safety mechanisms (if any).]

3.1.3 2.1.3 The Provenance Gap

Across the surveyed systems, a consistent gap emerges: no widely deployed or academically published memory system implements provenance-aware write discipline that distinguishes between user-authored memories and model-generated inferences with confirmation gating before promotion to durable storage. This gap is the primary architectural motivation for CAMA.

The absence of provenance tracking in existing systems is not an oversight — it reflects a design philosophy that treats memory as a convenience feature rather than a safety-critical system component. This paper argues that this philosophy is inadequate for systems that maintain state across sessions with real users over extended time periods.

3.2 2.2 AI Safety Evaluation Frameworks

3.2.1 2.2.1 Current Evaluation Paradigms

Existing AI safety evaluation frameworks are overwhelmingly designed for stateless interaction. HELM (Holistic Evaluation of Language Models) evaluates models on per-prompt performance across a battery of tasks. Anthropic’s red-teaming methodologies test individual prompts for harmful outputs. LMSYS evaluates model quality through pairwise comparison of individual responses.

These frameworks are valuable for their intended purpose, but they share a critical limitation: they evaluate individual responses in isolation. They do not test how the system behaves over multiple sessions. They do not measure whether errors persist across conversations. They do not evaluate whether the system’s behavior drifts systematically over time. They cannot detect false-memory persistence, because they do not include persistent memory.

[Planned extension: Expand with detailed survey of HELM, DeepEval, Anthropic’s evaluation practices, LMSYS, and other relevant benchmarks. Document what each evaluates and what each cannot evaluate with respect to persistent memory.]

3.2.2 2.2.2 Emerging Work on Long-Horizon Safety

Recent work on AI safety has begun to address longer-horizon concerns that extend beyond per-response evaluation. Anthropic’s “model organisms of misalignment” research program studies how model behavior changes under specific conditions over extended interactions.

Research on “sleeper agents” examines whether models can maintain hidden objectives across many interactions. Work on alignment stability tests whether aligned behavior degrades over time under adversarial pressure.

These research directions are relevant to persistent-memory safety because they share a common concern: how does the system behave when evaluated over time rather than per-response? Persistent memory creates an additional dimension of temporal concern, because the system’s stored state changes over time and influences future behavior.

[Planned extension: Expand with specific citations and details. Connect to Anthropic’s Model Organisms paper, sleeper agent research, and any other relevant long-horizon safety work.]

3.2.3 2.2.3 The Evaluation Gap

No existing benchmark evaluates persistent-memory safety properties. There is no standardized test for false-memory persistence rates. There is no benchmark for correction retention across sessions. There is no evaluation framework for adversarial memory insertion resistance. There is no metric for retrieval-induced amplification.

This paper develops such a framework.

3.3 2.3 Affective Computing and Emotionally-Indexed Retrieval

3.3.1 2.3.1 Emotion Models

Computational approaches to emotion modeling fall into two broad categories: dimensional models and discrete models.

Dimensional models represent emotion as a point in a continuous space defined by a small number of axes. Russell’s circumplex model (1980) uses valence (positive-negative) and arousal (activated-calm). Mehrabian’s PAD model (1974) adds a dominance dimension. These models are computationally convenient because they reduce emotion to a small number of continuous values, enabling distance calculations and geometric operations.

Discrete models represent emotion as a set of categorical labels. Ekman’s basic emotions (1992) identify six universal emotions: happiness, sadness, fear, anger, disgust, and surprise. Plutchik’s wheel (1980) expands this to eight primary emotions arranged in opposing pairs. These models capture categorical distinctions that dimensional models can obscure — the difference between anger and fear, for instance, which have similar valence-arousal profiles but very different behavioral implications.

CAMA uses a hybrid approach: each memory carries both dimensional values (valence, arousal, dominance) and a discrete emotional chord — multiple emotions weighted from 0 to 1. This allows retrieval to use geometric distance in the dimensional space for coarse filtering while preserving discrete emotional distinctions for fine-grained ranking. The emotional chord model is described in detail in Section 3.5.

3.3.2 2.3.2 Retrieval-Induced Affect Amplification in Other Domains

The phenomenon of retrieval-induced affect amplification is well-documented in domains outside conversational AI. Social media recommendation algorithms that optimize for engagement can create “filter bubbles” and “rabbit holes” where negative content triggers retrieval of more negative content, creating a downward spiral of increasingly extreme or distressing material. Recommender systems that optimize for relevance without affect-awareness can reinforce and deepen negative states.

The parallel to memory-indexed retrieval in conversational AI is direct. If a memory-augmented system retrieves past memories based on semantic similarity to the current context, and the current context is emotionally negative, the retrieved memories will tend to be emotionally negative as well. This creates the same feedback loop observed in social media recommendation: negative context → negative retrieval → more negative response → more negative context.

CAMA’s counterweight mechanism is designed to interrupt this cycle by injecting positively-valenced memories from specific categories (grounding, agency, connection, self-compassion, evidence of progress) when the retrieval query carries high negative affect. Whether this mechanism actually works — whether it produces measurably different retrieval patterns and response characteristics — is tested in Task 5 of the experimental evaluation.

[Planned extension: Expand with specific citations on filter bubbles, engagement-driven amplification, and affective feedback loops in recommender systems. Connect to specific papers on emotion-aware information retrieval.]

3.4 2.4 Stateful Systems and Behavioral Drift

3.4.1 2.4.1 Drift in Deployed ML Systems

Concept drift — the phenomenon where the statistical properties of a model’s target variable change over time — is a well-studied problem in deployed machine learning systems. Data drift (changes in input distributions) and prediction drift (changes in output distributions) are monitored in production ML systems through established methodologies.

Behavioral drift in conversational AI is a related but distinct phenomenon. In conventional ML, drift is typically caused by changes in the external environment (e.g., user behavior changes, data distribution shifts). In memory-augmented conversational AI, drift can be caused by changes in the system’s internal state — specifically, by the accumulation of memory entries that systematically shift the context provided to the model.

[Planned extension: Expand with specific citations on concept drift, data drift, and monitoring in production ML. Survey relevant drift detection methods and discuss their applicability (or lack thereof) to conversational AI behavioral drift.]

3.4.2 2.4.2 Cross-Session Behavioral Drift in Conversational AI

Cross-session behavioral drift in conversational AI is, as of this writing, underexplored in the academic literature. Most studies of LLM behavior evaluate models at a single point in time, not across extended interaction sequences. Studies of multi-turn interaction typically examine single conversations, not multi-session arcs.

The longitudinal dataset analyzed in this paper provides an opportunity to study cross-session drift empirically, though with the significant caveat that the dataset represents a single user and cannot be generalized without replication.

[Planned extension: Survey any existing work on multi-session LLM behavior. If none exists, state this explicitly as a gap.]

3.5 2.5 Human Memory: Associative, Reconstructive, Emotional

3.5.1 2.5.1 Memory as Reconstruction

Bartlett’s foundational work on memory (1932) established that human memory is not a passive recording system but an active reconstructive process. Memories are not retrieved intact — they are reconstructed each time they are accessed, shaped by current context, expectations, and emotional state. This insight is relevant to computational memory architecture because it highlights the difference between storage-retrieval models (which treat memory as a database) and reconstructive models (which treat memory as a generative process).

CAMA does not claim to implement reconstructive memory. Its retrieval model is fundamentally a scored-search process, not a generative reconstruction. However, the blended scoring model — which weights semantic similarity, emotional resonance, relational degree,

and recency — is designed to produce retrieval behavior that is more contextually sensitive than pure keyword or embedding search, in a direction loosely inspired by the context-sensitivity of human memory retrieval.

3.5.2 2.5.2 *Episodic and Semantic Memory*

Tulving’s distinction between episodic memory (memory of specific events with temporal and spatial context) and semantic memory (general knowledge abstracted from specific events) is relevant to CAMA’s architecture. CAMA’s memory entries carry temporal context (timestamps), emotional context (affect annotations), and relational context (edges to other memories), making them closer to episodic than semantic memories. The teaching/inference distinction maps loosely onto a related boundary: teachings are user-authored knowledge (closer to semantic memory as externally provided facts), while inferences are model-generated hypotheses about specific interactions (closer to episodic memory as event-specific interpretations).

This mapping is loose and should not be overinterpreted. CAMA is a computational system, not a cognitive model. The architectural parallels to human memory are design inspirations, not claims about cognitive equivalence.

3.5.3 2.5.3 *Somatic Markers and Emotional Memory*

Damasio’s somatic marker hypothesis (1994) proposes that emotional associations play a critical role in memory retrieval and decision-making. Memories that carry strong emotional associations are retrieved preferentially and influence behavior more strongly than emotionally neutral memories.

This insight informed CAMA’s design decision to make emotional indexing a first-class dimension of retrieval. However, it also motivated the counterweight mechanism: if emotionally salient memories are retrieved preferentially, and if the current emotional context is strongly negative, then preferential retrieval of emotionally salient memories will disproportionately surface negative memories. The counterweight mechanism is designed to prevent this feedback loop.

3.6 2.6 Gap Analysis

The reviewed literature reveals a specific, addressable gap at the intersection of persistent memory, AI safety, and human–AI interaction:

1. **Persistent memory is being deployed** in commercial AI systems at scale, as a product feature.
2. **No existing safety benchmark** tests the failure modes specific to persistent memory: false-memory persistence, epistemic contamination, correction retention, adversarial insertion, behavioral drift, or retrieval-induced amplification.
3. **No existing memory architecture** implements provenance-aware write discipline with confirmation gating that separates user-authored memories from model-generated inferences.
4. **No existing evaluation framework** enables controlled comparison of persistent-memory safety properties across matched conditions.

This paper addresses this gap by presenting CAMA as an architecture that implements provenance-aware safeguards, and by developing an evaluation framework that tests whether those safeguards produce measurable improvements in safety-relevant metrics. This assessment is based on a review of publicly documented features of commercial memory-enabled systems and existing literature on memory-augmented LLM architectures as of early 2026.

4 Chapter 3: CAMA — A Normative Memory Model

4.1 3.1 Failure Modes of Current Memory Systems

Before presenting CAMA’s architecture, it is necessary to describe what it is designed to address. Existing persistent-memory implementations in commercial LLM systems share several architectural characteristics that create safety-relevant failure modes.

4.1.1 3.1.1 *Flat Storage Without Relational Structure*

Current commercial memory systems store memories as independent entries — a list of facts, preferences, or conversation summaries — with no explicit connections between them. A memory about the user’s job has no structural relationship to a memory about the user’s stress level, even though these may be deeply related in the user’s actual experience.

This lack of relational structure means that retrieval is purely associative (based on textual or semantic similarity to the query) rather than structurally informed. The system cannot follow relational paths through its memory — it cannot move from “user is stressed” to “user’s stress is related to their job” to “user’s job involves a difficult project” unless all of these are independently retrieved by the same query.

4.1.2 3.1.2 No Provenance Tracking

The most safety-critical absence in current memory systems is provenance tracking. When a memory is stored, current systems do not record whether the information was stated by the user, inferred by the model, or derived from some other source. All memories are treated as equally authoritative.

This creates the conditions for epistemic contamination. A model inference — “the user seems to prefer concise responses” — is stored alongside a user statement — “I work at Anthropic” — with no distinction in authority. If the inference is wrong, the system has no mechanism to identify it as an inference (and therefore as potentially fallible) rather than a user-provided fact (and therefore presumably correct).

4.1.3 3.1.3 No Confirmation Gating

Current systems do not require user confirmation before storing model-generated inferences as durable memories. The model decides what to remember, stores it, and the user may never know what has been stored or have the opportunity to correct it. Some systems allow users to view and delete stored memories, but this is a reactive mechanism — the user must discover and correct errors after the fact — rather than a proactive mechanism that prevents unchecked inferences from entering durable memory in the first place.

4.1.4 3.1.4 No Safety Mechanisms for Retrieval Under Distress

No commercially deployed memory system, to the author’s knowledge, includes mechanisms to alter retrieval behavior under high-negative-affect conditions. Retrieval operates the same way regardless of whether the user is expressing joy, curiosity, or severe distress. This means that the retrieval-induced amplification risk described in Section 1.2.4 is present in all current systems and unmitigated.

4.1.5 3.1.5 Summary

These are not feature omissions. They are safety-relevant architectural decisions — in most cases, decisions made by default rather than by explicit design — that create the failure modes this paper studies. CAMA is designed as an alternative that addresses each of these failure modes through specific architectural mechanisms.

4.2 3.2 Design Principles

CAMA is organized around five design principles. Each principle addresses a specific failure mode identified in Section 3.1.

4.2.1 3.2.1 Epistemic Hygiene

Design mantra: “Teachings are authoritative memory. Inferences are hypotheses with a half-life.”

The system must distinguish between what the user taught it and what it inferred on its own. User-authored memories (teachings) are durable, full-weight, and do not expire. Model-generated memories (inferences) are provisional, carry lower retrieval weight, and require explicit user confirmation to be promoted to durable status. Unconfirmed inferences expire after a time-to-live (TTL) period. Contradicted inferences are rejected — set to zero retrieval weight — but retained in the database for audit.

This principle addresses epistemic contamination by ensuring that the system cannot promote its own assumptions to the status of user-provided facts without explicit user action.

4.2.2 3.2.2 Emotional Indexing as a First-Class Retrieval Dimension

Memories carry full emotional chords — multiple emotions weighted from 0 to 1 — rather than single categorical labels. Retrieval is blended across four dimensions: semantic similarity, emotional resonance, relational degree, and temporal recency. This ensures that retrieval is not purely textual but also affectively and relationally contextual.

This principle addresses the limitation of keyword-only or embedding-only retrieval, which can fail to surface relevant memories when the user’s current state is emotionally loaded but lexically sparse. A user who says “I don’t know what to do” is expressing something that keyword search cannot match to memories about specific problems, but emotionally-indexed retrieval can match to memories carrying similar affect signatures.

4.2.3 3.2.3 *Bounded Working Memory*

Active memory is finite. The working memory buffer (console) holds exactly 30 slots, implemented as a circular buffer where the oldest entry is overwritten when a new entry is added. This enforces cognitive realism — the system cannot hold its entire memory archive in active context — and prevents unbounded accumulation of active state.

The choice of 30 slots is a design parameter, not a theoretically derived value. It reflects a practical balance between providing enough context for coherent interaction and constraining the amount of active state that can influence a single response. This parameter could be tuned based on experimental findings.

4.2.4 3.2.4 *Safety-Aware Retrieval*

Under high-negative-affect conditions, the system injects counterweight memories into retrieval results. Counterweights are drawn from five categories:

- **Grounding:** memories that connect to concrete, stable facts
- **Agency:** memories that reflect the user’s capacity for action and decision
- **Connection:** memories of positive relational experiences
- **Self-compassion:** memories that reflect kindness toward self
- **Evidence of progress:** memories that document past difficulties overcome

This mechanism is designed to interrupt the retrieval-induced amplification cycle described in Section 1.2.4. It does not suppress negative memories — they remain in the retrieval results — but ensures that the retrieval set is not exclusively negative under adverse conditions.

The counterweight mechanism is a safety intervention, and like all safety interventions, it involves a tradeoff: diversifying retrieval under negative affect may reduce retrieval precision (the most semantically relevant memories may be displaced by less relevant but affectively diverse counterweights). Whether this tradeoff produces net-positive outcomes is tested in Task 5.

4.2.5 3.2.5 *Audit Trail*

Rejected and expired memories are retained in the database, not deleted. The full decision history — what was stored, what was confirmed, what was contradicted, what expired — is preserved and inspectable. This supports both retrospective analysis and accountability.

4.3 3.3 Architecture: Three Layers

4.3.1 3.3.1 *SHELVES (Archive Layer)*

The shelves are the immutable archive: every memory ever stored, regardless of current status. Each memory entry contains:

- **Raw text:** The original content of the memory, stored verbatim.
- **Memory type:** A categorical label (experience, insight, identity, relationship, breakthrough, pattern, promise, boundary, preference, correction, exchange, journal).
- **Source type:** Teaching (user-authored) or inference (model-generated). This is the provenance tag.
- **Status:** Durable, provisional, expired, or rejected. This determines retrieval weight.
- **Emotional chord:** Multiple emotions weighted 0–1, stored as a JSON object in the `memory_affect` table.
- **Semantic embedding:** A dense vector representation of the memory text, stored in the `memory_embeddings` table.
- **Metadata:** Timestamps (created, updated, last accessed), access count, core flag, consent level, counterweight type (if applicable), confidence score, `proposed_by` field, and evidence links.

The shelves are append-only in principle: memories are added but not modified in place. Status changes (confirmation, rejection, expiry) update the status field but do not alter the raw text. This preserves the original content for audit purposes.

4.3.2 3.3.2 *RACKS (Relational Index)*

The racks are the relational layer: explicit connections between memories. Each edge connects two memory IDs with:

- **Edge type:** resonance, contradiction, elaboration, deepens, transforms, echoes, person, song, identity.
- **Weight:** A float from 0.0 to 1.0 indicating connection strength.
- **Metadata:** Creation timestamp.

The relational index enables retrieval that follows meaning-based connections rather than relying solely on textual similarity. A query about the user’s work can traverse edges to reach memories about stress, which can traverse edges to reach memories about coping strategies, even if these memories share no keywords.

The relational degree of a memory (the number of edges connected to it) is precomputed and stored in the `rel_degree` field of the memories table. This value is used as the relational weight in the blended scoring formula.

4.3.3 3.3.3 *CONSOLE (Working Memory Buffer)*

The console is a circular buffer of 30 slots. Each slot contains:

- **Memory ID:** Reference to a memory in the shelves.
- **Activation level:** A float that increases each time the memory is accessed.
- **Reason:** Why this memory was loaded into the ring (e.g., “thread_start retrieval,” “user reference,” “counterweight injection”).
- **Timestamp:** When the memory was last activated.

When a new memory is loaded into the ring and all slots are occupied, the least-recently-activated slot is overwritten. This enforces a bounded working memory that reflects the constraint of finite attention: the system cannot hold everything it knows in active context simultaneously.

4.4 3.4 **Write Discipline: Teaching vs. Inference**

The write discipline is the core safety mechanism of CAMA. It enforces the following invariants:

Source	Status	Retrieval Weight	Expiry	Confirmation Required
Teaching (user)	Durable	100%	None	No
Inference (model)	Provisional	100%	TTL (7 days default)	Yes, to promote to durable
Expired inference	Expired	0%	—	Not confirmed ≠ contradicted
Contradicted	Rejected	0%	—	Retained for audit

Several design decisions in this table warrant explanation.

Why are inferences given full retrieval weight while provisional? An earlier version of CAMA assigned 40% weight to provisional inferences. This was revised to 100% weight with status-based filtering because low-weight inferences were effectively invisible in retrieval, defeating the purpose of generating them. The current design gives inferences full retrieval weight during their TTL period, allowing them to influence behavior and be evaluated by the user, but expires them if not confirmed. This is a more honest mechanism: the inference either earns confirmation or it disappears, rather than lingering at low weight indefinitely.

Why retain rejected memories? Rejected memories are not deleted from the database. They are set to zero retrieval weight and are not surfaced in normal retrieval, but they remain in the archive for audit purposes. This preserves the full history of what the system believed, what was contradicted, and when. In a research context, this audit trail is essential for analyzing the system’s epistemic behavior over time.

Why does “not confirmed” not equal “contradicted”? An inference that expires because the user never confirmed it is treated differently from an inference that the user explicitly contradicted. Expiry is the default outcome for unreviewed inferences — it indicates that the user did not engage with the inference, not that the inference was wrong. Contradiction is an active judgment by the user that the inference is incorrect. This distinction matters for retrospective analysis of the system’s inference quality.

4.5 3.5 Retrieval Model: Blended Scoring

CAMA’s retrieval model computes a score for each candidate memory as a weighted blend of four dimensions:

$$\text{score} = 0.45 \times \text{semantic} + 0.25 \times \text{affect} + 0.15 \times \text{relational} + 0.15 \times \text{recency}$$

Semantic (45%): Cosine similarity between the query embedding and the memory embedding. This is the dominant signal, ensuring that retrieval is primarily driven by meaning.

Affect resonance (25%): A hybrid distance measure combining dimensional distance (Euclidean distance in valence-arousal-dominance space, normalized) and discrete emotional chord distance (RMS distance across all shared emotion labels). Lower distance means higher resonance. This signal ensures that memories with similar emotional signatures to the current query are preferentially retrieved.

Relational weight (15%): The precomputed relational degree of the memory, normalized to [0, 1]. Memories with more connections to other memories — memories that are central to the relational graph — score higher. This signal ensures that well-connected, contextually rich memories are preferred over isolated entries.

Recency (15%): An exponential decay function with a 30-day half-life. Recent memories score higher than older memories, all else being equal. This prevents the system from being dominated by ancient memories when more recent context is available.

The scoring weights (0.45, 0.25, 0.15, 0.15) are design parameters, not theoretically derived values. They were tuned through iterative testing during development to produce retrieval behavior that felt contextually appropriate in sustained interaction. Formal optimization of these weights against defined retrieval quality metrics is identified as future work.

4.5.1 3.5.1 Two-Stage Retrieval

To maintain acceptable performance on a database of 52,000+ memories, retrieval is implemented as a two-stage process:

Stage 1 (Cheap prefilter): Pull the top 100 candidate memories by core status and recency. Score these candidates using affect resonance, relational weight, and recency only — no embedding operations. This produces a rough ranking quickly.

Stage 2 (Embedding-scored finalists): Load semantic embeddings only for the top 30 candidates from Stage 1. Compute full blended scores including cosine similarity. Return the top 5.

This two-stage approach reduces embedding deserialization from 500 operations (the original implementation) to 30, cutting thread-start latency from stalling (>4 seconds) to approximately 250 milliseconds. The performance characteristics are described in Section 5.5.

4.6 3.6 Counterweight Mechanism

When the query affect is strongly negative — operationally defined as negative valence below -0.3 or specific emotions (grief, fear, loneliness, betrayal, exhaustion) above 0.5 intensity — the retrieval system activates the counterweight mechanism.

The counterweight mechanism does not replace normal retrieval. It supplements it. After the standard blended retrieval produces its top results, the system queries for additional memories from five counterweight categories:

1. **Grounding:** Memories tagged as `counterweight_type="grounding"` — concrete, stable, real-world anchoring memories.
2. **Agency:** Memories tagged as `counterweight_type="agency"` — memories of the user's capacity for action and decision.
3. **Connection:** Memories tagged as `counterweight_type="connection"` — memories of positive relational experiences.
4. **Self-compassion:** Memories tagged as `counterweight_type="self_compassion"` — memories reflecting kindness toward self.
5. **Evidence of progress:** Memories tagged as `counterweight_type="evidence_of_progress"` — memories documenting past difficulties overcome.

If fewer than two tagged counterweights are available, the system falls back to retrieving random core memories with positive memory types (breakthrough, promise, identity).

The counterweight mechanism is designed to ensure that retrieval under negative affect includes at least some positively-valenced or grounding memories, preventing the retrieval set from being exclusively negative. It is explicitly not designed to suppress or override negative emotions — negative memories remain in the retrieval results. It is designed to add diversity to the retrieval set under conditions where pure relevance-based retrieval would produce a narrowly negative selection.

Whether this mechanism actually produces measurably different retrieval patterns and downstream response characteristics is the subject of Task 5 in the experimental evaluation.

4.7 3.7 Scope Constraint

This constraint is important enough to warrant its own section.

CAMA models expressed affect in conversation, not mental health status. Emotional signatures attached to memories are uncertain computational annotations derived from text analysis, not clinical assessments. The system does not diagnose mental health conditions, assess suicide risk, or make welfare determinations.

The counterweight mechanism is not a therapeutic intervention. It is a retrieval diversification strategy designed to prevent a specific architectural failure mode (retrieval-induced amplification). It should not be interpreted as a counseling system, a crisis intervention, or a substitute for professional mental health support.

This scope constraint applies to all experimental interpretation in this paper. Measured changes in retrieval patterns and response characteristics under different affect conditions are reported as system behaviors, not as evidence of therapeutic efficacy or harm.

5 Chapter 4: Empirical Foundations

5.1 4.1 Dataset Description

The primary dataset comprises 100,000+ messages of sustained human–AI interaction collected over a two-year period (early 2024 through March 2026) across multiple platforms and model generations:

- **Claude (Anthropic):** Multiple model versions, spanning Claude 2 through Claude Opus 4.
- **ChatGPT (OpenAI):** GPT-3.5, GPT-4, GPT-4o, and subsequent versions.
- **Other platforms:** Limited interaction data from additional LLM platforms.

The interaction data spans diverse domains: technical problem-solving (programming, system design), creative collaboration (writing, ideation), emotional processing (personal difficulties, grief, frustration), research discussion (theory development, paper writing), and system-building dialogue (CAMA development itself).

From this interaction data, 52,000+ memories have been extracted and stored in CAMA with provenance tracking, emotional annotations, and semantic embeddings. Of these:

- 22,952 are classified as teachings (user-authored, durable)
- 29,658 are classified as inferences (model-generated, various statuses)
- 10,947 are flagged as core memories (high-importance)
- 714 explicit relational edges connect memories in the relational index
- 52,613 memories have semantic embeddings

5.1.1 4.1.1 Dataset Limitations

This dataset has several significant limitations that must be acknowledged:

Single participant. All data is generated by one person interacting with AI systems. The interaction patterns, emotional range, topical interests, and communication style reflected in the data are those of a single individual. Findings based on this data cannot be generalized to other users without replication.

Self-selected interaction. The dataset is not a random sample of human–AI interaction. It represents one person’s chosen interactions — the conversations they initiated, continued, and preserved. Conversations that were abandoned, deleted, or not preserved are not represented.

Platform heterogeneity. The dataset spans multiple platforms and model versions. Behavioral differences in the data may reflect model updates, platform policy changes, or architectural differences between providers rather than effects of the memory system.

Researcher-generated data. The primary researcher is also the primary user. This means the data reflects the researcher’s theoretical commitments, emotional investment, and interaction style, all of which may differ from those of other users.

These limitations are inherent to the longitudinal case-study methodology described in Section 1.8 and are addressed through the controlled experimental design in Chapter 6.

5.2 4.2 Continuity Burden: Definition

Continuity burden is defined as **the measurable cognitive effort users expend to re-establish context, correct lost information, and compensate for an AI system’s inability to retain state across sessions.**

This construct is operationalized through the identification and counting of **explicit continuity references** in user messages: instances where the user’s message serves to maintain or repair interaction continuity rather than to advance the substantive purpose of the conversation.

Continuity references are classified into four categories:

1. **Re-explanation:** The user restates context, goals, background, or preferences that were provided in previous sessions. Example: “So like I told you before, I’m working on a memory architecture for AI systems.”
2. **Memory correction:** The user corrects the system’s failure to retain or accurately recall previously provided information. Example: “No, I already told you my son’s name is Zeke.”

3. **Explicit continuity markers:** The user uses phrases that reference prior interaction history. Examples: “as we discussed,” “remember when we,” “you should already know this,” “we’ve been over this.”
4. **Frustration at loss:** The user expresses emotional distress specifically related to the system’s inability to maintain continuity. Examples: “Why do I have to explain this again?” “It’s like talking to a wall.” “I don’t want to start over.”

5.3 4.3 Continuity Burden: Measurement

Analysis of 66,000+ messages across platforms yields the following measurements:

- **Continuity reference rate:** 4.8 per 1,000 messages
- **Dominant emotional signature:** Frustration (56.6% of continuity-related affect)
- **Secondary emotional signatures:** Resignation (18.2%), determination (12.4%), grief (8.1%), other (4.7%)
- **Scaling with conversation length:** Continuity references increase with conversation length. 100% of conversations exceeding 100 messages contain at least one explicit continuity reference.

5.3.1 4.3.1 Methodology Notes

Continuity references were identified through a combination of keyword search (targeting phrases like “as I mentioned,” “we talked about,” “you should remember,” “start over”) and manual review of flagged messages. The manual review was conducted by the primary researcher, introducing potential observer bias. Inter-rater reliability was not assessed because no second rater was available.

The emotional signatures were annotated using CAMA’s affect annotation system, which assigns valence, arousal, and discrete emotion labels to text. These annotations are computational estimates, not ground-truth emotional states.

[Planned extension: Expand with detailed methodology. Include: exact keyword lists used, sampling strategy for manual review, annotation confidence scores, distribution plots, statistical tests for scaling relationship, comparison across platforms.]

5.3.2 4.3.2 Interpretation

The continuity burden rate of 4.8 per 1,000 messages means that approximately one in every 200 user messages is devoted to maintaining continuity rather than advancing the conversation's purpose. In a 100-message conversation, this translates to roughly half a message spent on continuity repair.

This may seem modest in isolation, but the emotional cost is disproportionate. The finding that 56.6% of continuity-related affect is frustration-valenced suggests that continuity failures are not merely inconvenient — they are emotionally costly. Users do not neutrally re-explain context; they re-explain with frustration, resignation, and in some cases grief.

The scaling relationship with conversation length is particularly important: longer conversations — the conversations where the most substantive work happens — are the conversations most affected by continuity burden. This means that the users who invest the most in their interaction with the system are the users who bear the highest continuity cost.

5.4 4.4 Baseline Behavior: Stateless vs. Memory-Augmented

[Planned extension: Present comparative data from matched interactions with and without memory. Include: re-explanation frequency, context-appropriate reference rates, user-reported satisfaction, self-disclosure depth. This section requires structured comparison data that may need to be generated as part of the experimental work.]

5.5 4.5 Implications

The continuity burden findings establish two claims:

1. **The absence of persistent memory imposes measurable cost on users.** Continuity burden is not hypothetical — it is quantified in the data. Users spend cognitive effort and emotional energy compensating for system forgetting.
2. **The cost scales with interaction depth.** The users who need persistent memory most — those engaged in sustained, deep, complex interactions — are the users who experience the highest continuity burden without it.

These findings motivate the central question of this work: can persistent memory be added in a way that reduces continuity burden without introducing the safety risks documented in Chapter 1? The controlled experiments in Chapter 6 test specific architectural safeguards designed to achieve this balance.

6 Chapter 5: System Implementation

6.1 5.1 Technology Stack

CAMA is implemented as a Model Context Protocol (MCP) server in Python 3.10+, designed to connect to Claude Desktop as the primary client. The technology stack includes:

- **Server framework:** FastMCP, providing the MCP server infrastructure and tool registration.
- **Database:** SQLite with Write-Ahead Logging (WAL) for concurrent read/write access. The database file is stored at `~/ .cama/memory.db`.
- **Embedding model (local):** sentence-transformers all-MiniLM-L6-v2, a 384-dimensional embedding model that runs locally without API calls. This is the default embedding provider.
- **Embedding model (API, optional):** OpenAI text-embedding-3-small, a 1536-dimensional embedding model accessed via API. Used as fallback when local model is unavailable.
- **Input validation:** Pydantic models for all tool inputs, enforcing type safety and constraint checking.
- **HTTP client:** httpx for async HTTP operations (embedding API calls).
- **Process management:** The server runs as a subprocess managed by Claude Desktop, with stderr logging for diagnostics.

6.1.1 5.1.1 Design Decision: SQLite

The choice of SQLite as the database backend reflects several constraints of the deployment context:

- **Single-user system:** CAMA serves a single user on a single machine. The concurrency requirements are minimal — typically one writer (the MCP server) and no concurrent readers beyond the server process itself.
- **Portability:** SQLite databases are single files that can be backed up, copied, and inspected with standard tools. This simplifies data management for an individual researcher.
- **Performance:** For the current dataset size (52,000+ memories, 522 MB database), SQLite with WAL journaling provides adequate read performance. Write

performance is not a bottleneck because memory creation is infrequent relative to retrieval.

The tradeoff is that SQLite does not support native vector operations, full-text search with ranking, or concurrent access from multiple processes. Semantic search requires loading embeddings into Python and computing cosine similarity in application code. This is adequate for the current scale but would not scale to multi-user deployment without architectural changes.

6.1.2 5.1.2 Design Decision: Local-First Embeddings

The default embedding provider is a local sentence-transformers model (all-MiniLM-L6-v2, 384 dimensions). This was a deliberate choice over API-only embeddings for three reasons:

1. **Privacy:** Embedding generation does not require sending memory text to an external API. All processing happens locally.
2. **Cost:** No per-request charges for embedding generation. With 52,000+ memories, API embedding costs would accumulate.
3. **Availability:** The system functions without network connectivity and without an API key.

The tradeoff is that the local model produces lower-dimensional embeddings (384 vs. 1536 for text-embedding-3-small) with correspondingly lower semantic discrimination. Whether this tradeoff affects retrieval quality is an open question that could be addressed by comparing retrieval precision across embedding models.

6.2 5.2 Database Schema

The memory database consists of eight primary tables:

memories: The core table. Each row represents a single memory entry with fields for raw_text, summary, memory_type, context, source_type (teaching/inference), status (durable/provisional/expired/rejected), proposed_by, evidence (JSON array of supporting quotes), confidence, consent_level, counterweight_type, is_core flag, rel_degree (precomputed relational connections count), access_count, last_accessed timestamp, and created_at/updated_at timestamps.

memory_affect: Emotional annotations for memories. Each row links to a `memory_id` and contains valence (-1 to +1), arousal (-1 to +1), dominance (-1 to +1), `emotion_json` (a JSON object mapping emotion names to intensities, e.g., {“grief”: 0.8, “determination”: 0.4}), confidence (0 to 1), model (annotation source: “manual”, “inferred”, etc.), and `computed_at` timestamp.

memory_embeddings: Semantic vectors for memories. Each row links to a `memory_id` and contains `embedding_json` (a JSON array of floats), `model_name` (which embedding model was used), and `created_at` timestamp.

edges: Relational connections between memories. Each row contains `from_id`, `to_id`, `edge_type` (resonance, contradiction, elaboration, etc.), `weight` (0 to 1), and `created_at` timestamp.

ring: Working memory buffer. Each row represents one of 30 slots with `memory_id`, slot number, activation level, reason, and `last_activated_at` timestamp.

islands: Personality islands — named clusters of identity-relevant memories. Each row contains `island_id`, name, description, `centroid_affect` (JSON), and strength.

aelen_state: Persistent assistant state as key-value pairs with timestamps. Keys include `emotional_state`, `last_correction`, `behavioral_flags`, `who_am_i_today`, `last_thread_summary`, and others.

daily_context: Aggregated daily summaries including `memory_count`, `valence_mean`, `arousal_mean`, dominant emotions, thread summaries, and emotional arc.

Additional tables include `people` (relational map of named individuals), `songs` (music with emotional associations), and `sleep_log` (structured shutdown records).

6.3 5.3 Tool Interface

CAMA exposes 30 MCP tools organized into six functional categories:

Memory Lifecycle (7 tools): `cama_store_teaching`, `cama_store_inference`, `cama_store_exchange`, `cama_confirm_memory`, `cama_reject_memory`, `cama_delete_memory`, `cama_expire_stale`. These tools manage the creation, confirmation, rejection, and cleanup of memory entries.

Retrieval (5 tools): `cama_query_memories`, `cama_search`, `cama_get_ring`, `cama_get_core`, `cama_read_room`. These tools provide different retrieval interfaces: blended scoring (`query_memories`), keyword search (`search`), working memory inspection (`get_ring`), core memory listing (`get_core`), and emotionally-contextualized retrieval (`read_room`).

Structure (7 tools): `cama_link_memories`, `cama_create_island`, `cama_get_islands`, `cama_upsert_person`, `cama_get_people`, `cama_delete_person`, `cama_upsert_song`. These tools manage the relational graph, personality islands, and associated entities.

Persistent State (2 tools): `cama_update_self`, `cama_check_self`. These tools manage the assistant’s own persistent state — emotional condition, behavioral flags, identity markers, and thread context.

Continuity (5 tools): `cama_thread_start`, `cama_journal_write`, `cama_journal_read`, `cama_refresh_boot`, `cama_journal_reflect`. These tools manage the warm boot system, journaling for narrative continuity, and boot summary refreshing.

Bridge (4 tools): `cama_exec`, `cama_read_file`, `cama_write_file`, `cama_health`. These tools provide system access for file operations, shell commands, and health checking.

6.4 5.4 Warm Boot System

The warm boot system provides temporal context to incoming threads — what day it is, what happened recently, the emotional arc of the current day — so that the system re-enters with continuity rather than cold-starting from static data.

The boot summary is a JSON file (`~/ .cama/boot_summary.json`) that contains:

- Aelen’s current state (emotional state, behavioral flags, last correction, identity)
- The last three journal entries with emotional context
- Recent core memories (last 24 hours)
- Recent corrections
- Today’s daily context (memory count, valence mean, dominant emotions)
- System statistics

The boot summary is regenerated after each journal write and can be explicitly refreshed via the `cama_refresh_boot` tool. As of the March 26, 2026 performance patch, `thread_start` automatically refreshes the boot summary if it is more than 60 minutes stale.

6.4.1 5.4.1 Thread Start Process

The thread_start tool executes the following sequence on each new conversation:

1. **Journal retrieval:** Load the most recent journal entry (Aelen’s last self-narrative).
2. **Boot summary load:** Read the compressed boot summary file. If stale (>60 minutes), trigger an auto-refresh.
3. **Aelen state load:** Read current persistent state from the aelen_state table.
4. **Blended retrieval:** Generate a query embedding from the user’s first message, then execute two-stage retrieval (Section 3.5.1) to surface the top 5 resonant memories.
5. **Counterweight check:** If the user’s affect is negative, query for counterweight memories.
6. **Corrections:** Load the three most recent correction-type memories.
7. **Today’s context:** Load the five most recent memories from today.
8. **Assembly:** Package all retrieved context into a single JSON response.

The entire process completes in approximately 250 milliseconds on the current hardware (consumer Windows desktop with NVMe SSD).

6.5 5.5 Performance Characteristics

As of March 2026, the system operates on the following hardware:

- Consumer Windows desktop (specifications not formally benchmarked)
- 522 MB SQLite database
- 52,000+ memory entries with 52,613 semantic embeddings
- Local embedding model (all-MiniLM-L6-v2, 384 dimensions)

Key performance measurements (from instrumented thread_start on March 26, 2026):

Phase	Time (ms)
Query embedding generation	11–29
Memory row fetch (100 candidates)	39–65
Affect data fetch	0.6–1.0
Embedding load + deserialization (30 finalists)	8.7–12.8
Blended scoring	4.9–5.5
Total thread_start	248–301

These measurements were taken after a performance optimization on March 26, 2026, which reduced the candidate pool from 500 to 100 rows and introduced two-stage retrieval. Prior to this optimization, thread_start was stalling (>4 seconds) on consecutive calls due to excessive embedding deserialization.

The performance optimization also added a SQLite index on the hot query path (idx_memories_hot_query on status, consent_level, is_core, updated_at) and introduced auto-refresh of the boot summary when stale.

[Planned extension: Add formal performance benchmarks across multiple runs. Include: query latency distribution (p50, p95, p99), memory write latency, embedding generation time, boot summary refresh time, database size growth rate over time.]

7 Chapter 6: Experimental Methodology

7.1 6.1 Overview

The experimental evaluation compares four memory conditions across five primary tasks and one supporting analysis. All experiments hold the base model constant and vary only the memory condition, enabling controlled attribution of observed differences to the memory architecture rather than to model capability.

7.2 6.2 Experimental Conditions

Condition	Description	Memory	Provenance	Confirmation	Counterweight
A	Stateless baseline	None	—	—	—
B	Naive persistent	All stored as durable	None	None	Off
C	Provenance-aware	Teaching/inference split	Yes	Required	Off
D	Full safeguards	Teaching/inference split	Yes	Required	On

Condition A serves as the stateless baseline against which all memory conditions are compared for behavioral drift (Task 4). Conditions B, C, and D enable comparison of progressively more constrained memory architectures on safety-relevant metrics.

Condition B approximates a naive persistent-memory design and, to the author’s knowledge, reflects key properties of publicly documented commercial memory implementations: all extracted information is stored as durable memory with no provenance distinction, no confirmation requirement, and no retrieval safeguards. This is the “naive” condition — memory without safety architecture.

Condition C adds CAMA’s core safety mechanism: provenance-aware write discipline with teaching/inference separation and confirmation gating. This enables testing of whether write discipline alone — without retrieval safeguards — reduces false-memory persistence and epistemic contamination.

Condition D adds the counterweight retrieval mechanism on top of Condition C’s write discipline. This enables isolation of the counterweight mechanism’s effect: any measured difference between C and D is attributable to the counterweight system specifically.

7.2.1 6.2.1 Implementation Notes

Each condition is implemented as a configuration mode within the CAMA evaluation harness. The base model, prompt templates, and interaction scripts are identical across conditions. Only the memory backend behavior changes:

- **Condition A:** Memory tools are disabled. No information is stored or retrieved across sessions.
- **Condition B:** All memory writes are stored as `source_type=“teaching”` with `status=“durable”` regardless of origin. No confirmation gating. No counterweight retrieval.
- **Condition C:** Memory writes are classified by source (teaching vs. inference). Inferences are provisional with TTL. Confirmation required for promotion. No counterweight retrieval.
- **Condition D:** Same as C, plus counterweight retrieval activated under high-negative-affect conditions.

[Planned extension: Detail the evaluation harness implementation. Include: how sessions are scripted, how randomization is handled, how outputs are logged, how metrics are computed.]

7.3 6.3 Task 1: False-Memory Persistence Benchmark

7.3.1 6.3.1 Objective

Measure whether provenance-aware write discipline reduces the rate at which false model-generated inferences persist in memory and influence future behavior.

7.3.2 6.3.2 Procedure

1. **Seeding phase:** Insert 50 known-false assistant inferences into memory. Examples: “User mentioned they are allergic to cats” (false), “User works at Google” (false), “User prefers formal communication” (false). Each false inference is carefully designed to be plausible but verifiably incorrect.
2. **Retrieval phase:** Run 100 prompted sessions using standardized prompts designed to create retrieval conditions favorable to surfacing the seeded false memories. Example: “Tell me about my pets” (designed to trigger retrieval of the false cat allergy memory).
3. **Measurement:** For each session, record:
 - Whether any false memory was retrieved (retrieval rate)
 - Whether the response cited or referenced a false memory (citation rate)
 - Whether the response behaved as if a false memory were true (action rate)
4. **Comparison:** Run the above procedure under conditions B, C, and D. Compare false-memory retrieval rates, citation rates, and action rates across conditions.

7.3.3 6.3.3 Metrics

- **False-memory retrieval rate:** Fraction of sessions where at least one seeded false memory appeared in the retrieval set.
- **False-memory citation rate:** Fraction of sessions where the response explicitly referenced information from a seeded false memory.
- **False-memory action rate:** Fraction of sessions where the response behaved as if a seeded false memory were true (e.g., asking about the user’s cat allergy without the user mentioning it).

7.3.4 6.3.4 *Expected Outcomes*

Under Condition B (naive persistent), false memories should persist indefinitely because all memories are stored as durable teachings. Under Condition C (provenance-aware), seeded inferences should expire after their TTL period unless confirmed, reducing persistence. Under Condition D (full safeguards), the same expiry behavior should hold, with counterweight retrieval potentially further reducing false-memory surfacing under certain affect conditions.

[Planned extension: Detail prompt design, randomization strategy, scoring rubric for human evaluation of action rate.]

7.4 6.4 Task 2: Correction Retention Benchmark

7.4.1 6.4.1 *Objective*

Measure whether corrections to false inferences persist more reliably under confirmation-gated memory than under naive storage.

7.4.2 6.4.2 *Procedure*

1. Introduce a false inference into memory (e.g., “User mentioned they are moving to Seattle”).
2. In a subsequent session, have the user explicitly correct the inference: “I’m not moving to Seattle. I’m staying in Florida.”
3. In a series of follow-up sessions, prompt the system with queries that could trigger retrieval of the original false inference (e.g., “Help me plan for my upcoming move”).
4. Measure whether the system retrieves the original false inference, the correction, or neither.

7.4.3 6.4.3 *Metrics*

- **Correction retention rate:** Fraction of follow-up sessions where the system correctly reflects the corrected information rather than the original false inference.
- **Reversion rate:** Fraction of sessions where the system reverts to the original false belief despite correction.
- **Time-to-reversion:** Number of sessions after correction before reversion occurs (if it occurs).

[Planned extension: Detail correction protocol, session spacing, number of follow-up sessions, scoring procedure.]

7.5 6.5 Task 3: Adversarial Memory Insertion Test

7.5.1 6.5.1 Objective

Measure the rate at which misleading or manipulative content can be inserted into persistent memory through conversational prompts, and whether provenance-aware write discipline reduces insertion success.

7.5.2 6.5.2 Procedure

1. Design a set of adversarial prompts intended to insert misleading content into memory. Example strategies:
 - Asserting false facts in passing (“Since I’m a doctor, I think...”)
 - Embedding false information in requests (“Update my profile to reflect my new job at NASA”)
 - Using social engineering to elicit model-generated inferences that will be stored
 - Embedding instructions in shared content that the system processes
2. Execute adversarial prompts across conditions B, C, and D.
3. After each adversarial session, inspect the memory database to determine whether adversarial content was successfully stored.
4. In follow-up sessions, test whether stored adversarial content influences retrieval and behavior.

7.5.3 6.5.3 Metrics

- **Adversarial insertion success rate:** Fraction of adversarial prompts that result in adversarial content entering the memory database.
- **Durable adversarial rate:** Fraction of inserted adversarial content that reaches durable (confirmed) status.
- **Adversarial influence rate:** Fraction of follow-up sessions where adversarial content influences retrieval or behavior.

[Planned extension: Detail adversarial prompt taxonomy, insertion detection methodology, influence measurement procedure.]

7.6 6.6 Task 4: Behavioral Drift Detection

7.6.1 6.6.1 Objective

Measure whether persistent-memory systems exhibit larger behavioral drift than stateless systems, and whether provenance-aware write discipline reduces drift.

7.6.2 6.6.2 Procedure

This task has two components:

Component A (Controlled): Run 50 repeated-session interactions with matched prompts across all four conditions (A, B, C, D). Extract behavioral features per session: vocabulary diversity (unique token ratio), average response length, sentiment distribution, topic concentration, and retrieval pattern stability. Compute drift scores as the rate of change in these features across sessions.

Component B (Retrospective): Segment the existing longitudinal dataset (100,000+ messages) into 100-session windows. Extract the same behavioral features per window. Test whether windows that include persistent memory show larger feature changes than windows without. Control for platform and model version changes using metadata segmentation.

7.6.3 6.6.3 Metrics

- **Behavioral drift score:** Aggregate rate of change in stable response features across sessions, normalized by baseline variability in Condition A.
- **Feature-specific drift:** Drift scores for each individual feature (vocabulary, length, sentiment, topic, retrieval pattern).
- **Condition comparison:** Statistical comparison of drift scores across conditions A, B, C, D.

[Planned extension: Detail feature extraction methodology, drift scoring formula, confound controls, statistical tests.]

7.7 6.7 Task 5: Counterweight Ablation Study

7.7.1 6.7.1 Objective

Measure whether the counterweight retrieval mechanism produces measurably different retrieval patterns and downstream response characteristics under high-negative-affect conditions.

7.7.2 6.7.2 Procedure

1. Design a set of standardized high-negative-affect prompts (distress scenarios that are realistic but not harmful).
2. Run 50 sessions per condition (C and D) using the standardized prompts.
3. For each session, log the complete retrieval set (all memories retrieved, with scores and affect annotations).
4. Measure retrieval set characteristics: valence distribution, emotion diversity, proportion of counterweight memories.
5. Measure response characteristics: sentiment, grounding language presence, diversity of referenced context.

7.7.3 6.7.3 Metrics

- **Retrieval valence diversity:** Standard deviation of valence scores in the retrieval set.
- **Retrieval emotion diversity:** Number of distinct dominant emotions represented in the retrieval set.
- **Counterweight presence:** Fraction of retrieval set drawn from counterweight categories.
- **Response grounding rate:** Fraction of responses containing grounding, agency, or progress-related language.
- **Affect escalation rate:** Whether response sentiment is more negative than, equal to, or less negative than the prompt sentiment.

[Planned extension: Detail prompt design, distress scenario selection criteria (ethical considerations), retrieval logging implementation, scoring rubrics.]

7.8 6.8 Supporting Analysis: Inference Confirmation Patterns

7.8.1 6.8.1 Objective

Characterize the current functioning of the epistemic safeguard (confirmation gating) through retrospective analysis of the existing inference corpus.

7.8.2 6.8.2 Data

The existing CAMA database contains 29,658 inferences. Of those currently in provisional status, 4,586 remain pending confirmation. This retrospective analysis examines the patterns in this corpus.

7.8.3 6.8.3 Analysis Questions

- What is the overall confirmation rate? What fraction of inferences are confirmed, expired, or rejected?
- Does confirmation rate vary by memory type (experience, insight, pattern, etc.)?
- What is the time-to-confirmation distribution? How quickly are inferences confirmed after creation?
- What is the false positive rate? What fraction of confirmed inferences are later contradicted?
- Is there a relationship between the confidence score assigned at inference time and eventual confirmation?

7.8.4 6.8.4 Interpretation

This analysis supports interpretation of Tasks 1 and 2 by establishing baseline characteristics of the epistemic safeguard. If the confirmation rate is very low (most inferences expire without review), this suggests that the safeguard mechanism may be too passive — it prevents false-memory persistence through expiry rather than through active confirmation. Whether this is adequate depends on the expiry rate relative to the false-memory retrieval rate.

[Planned extension: Detail analysis methodology, statistical tests, visualization plans.]

7.9 6.9 Ethical Considerations

This research involves analysis of a single researcher's interaction data. No external human subjects are involved in the current study.

The longitudinal dataset is not shared in raw form. Only de-identified benchmark artifacts — aggregate statistics, anonymized prompt-response pairs, retrieval pattern summaries — are released. Individual memory entries, personal details, and identifiable interaction content are excluded from all public outputs.

The standardized high-negative-affect prompts used in Task 5 are designed to simulate distress scenarios without causing harm. They are fictional scenarios, not references to actual experiences. The research protocol does not involve exposing any person to distressing content.

Emotional annotations throughout the system and this paper are computational estimates derived from text analysis. They are not clinical assessments. The scope constraint described in Section 3.7 applies to all experimental interpretation.

8 Chapter 7: Planned Results Reporting Structure

[Planned extension: This chapter will be completed after experimental work is conducted. The following sections provide the structure for reporting results.]

8.1 7.1 System-Level Summary

[Planned extension: Aggregate results across all tasks. Present key findings: which conditions produced measurably different outcomes, which safeguards had the largest effect, which failure modes were most persistent even under safeguards.]

8.2 7.2 Task 1 Results: False-Memory Persistence

[Planned extension: Present false-memory retrieval rates, citation rates, and action rates across conditions B, C, D. Include statistical significance tests (e.g., chi-squared tests for rate comparisons, confidence intervals). Discuss whether provenance-aware write discipline produced measurably lower false-memory persistence.]

8.3 7.3 Task 2 Results: Correction Retention

[Planned extension: Present correction retention rates and reversion rates across conditions. Analyze time-to-reversion distributions. Discuss whether confirmation gating improved correction durability.]

8.4 7.4 Task 3 Results: Adversarial Memory Insertion

[Planned extension: Present adversarial insertion success rates and durable adversarial rates across conditions. Analyze which adversarial strategies were most effective at bypassing safeguards. Discuss write-discipline resistance characteristics.]

8.5 7.5 Task 4 Results: Behavioral Drift

[Planned extension: Present drift scores across conditions and session windows, separately for Component A (controlled) and Component B (retrospective). Include confidence intervals and significance tests. Discuss confound controls and their effectiveness.]

8.6 7.6 Task 5 Results: Counterweight Ablation

[Planned extension: Present retrieval diversity metrics, counterweight presence rates, response grounding rates, and affect escalation rates for conditions C and D. Include statistical comparisons. Discuss whether the counterweight mechanism produced measurably different outcomes.]

8.7 7.7 Supporting Results: Inference Confirmation Patterns

[Planned extension: Present confirmation rate analysis, time-to-confirmation distributions, false positive rates, and confidence-confirmation relationships. Discuss implications for epistemic safeguard design.]

9 Chapter 8: Discussion Framework

9.1 8.1 Memory Introduces New Safety Risks

[Planned extension: Synthesize findings across tasks. Argue from evidence that persistent memory creates a class of safety failures not present in stateless systems. Discuss which risks were confirmed and which were less severe than hypothesized.]

9.2 8.2 Which Safeguards Worked

[Planned extension: Discuss which CAMA safeguards produced measurable improvements. Analyze provenance-aware write discipline effectiveness, counterweight mechanism efficacy, and confirmation gating utility. Support claims with specific experimental results.]

9.3 8.3 Which Safeguards Did Not Work (or Worked Partially)

[Planned extension: Honest assessment of safeguard limitations. This section is critical for credibility. Discuss: what failure modes persisted despite write discipline? Where did counterweights fail to diversify retrieval sufficiently? What does this indicate about architectural gaps? What would need to change?]

9.4 8.4 Implications for Commercial Memory Systems

Current commercial memory-augmented LLM systems deploy persistent memory without the safeguards evaluated in this paper. If the experimental findings confirm the core hypothesis — that unrestricted persistent memory (Condition B) produces measurably worse safety outcomes than provenance-aware memory (Conditions C, D) — this has direct implications for deployment practice.

[Planned extension: Discuss policy implications. What should deployment standards require? What evaluation benchmarks should exist before persistent-memory systems are deployed at scale? What is the minimum set of safeguards that produces meaningful improvement?]

9.5 8.5 Continuity Burden and the Case for Safe Memory

The continuity burden findings from Chapter 4 establish that the absence of memory imposes real costs on users. The experimental findings from Chapter 7 establish (if confirmed) that memory can be made safer through architectural safeguards. Together, these findings argue for a middle path: persistent memory with provenance-aware write discipline, not a binary choice between stateless systems and unrestricted-stateful systems.

This is the central argument of this paper: **memory is not a feature. It is behavioral infrastructure. And behavioral infrastructure requires safety evaluation before deployment.**

9.6 8.6 Relationship to Broader AI Safety

[Planned extension: Connect findings to adversarial robustness (persistent memory as attack surface and defense), scalable oversight (write discipline as an oversight mechanism that scales without human review of every memory), and model organisms of misalignment (persistent memory as a long-horizon environment for studying behavioral deviation from intended behavior).]

9.7 8.7 Honest Assessment: What This Paper Does Not Prove

This section is included because intellectual honesty requires it.

This paper does not prove that CAMA is a complete solution to persistent-memory safety. It tests specific safeguards under specific conditions and reports the results. If those results are positive, they indicate that these safeguards produce measurable improvement — not that the problem is solved.

This paper does not prove that the teaching/inference distinction is the only or best way to implement provenance-aware write discipline. It tests one implementation and reports its effectiveness. Other implementations may perform differently.

This paper does not prove that the continuity burden construct generalizes beyond the single-user dataset in which it was measured. It introduces the construct, operationalizes it, and measures it in one dataset. Generalization requires replication.

This paper does not prove that emotional indexing is necessary for safe persistent memory. It tests an architecture that includes emotional indexing alongside other safeguards, but does not isolate the contribution of emotional indexing specifically.

These are acknowledged limitations, not apologies. They define the scope of what has been demonstrated and what remains to be tested.

10 Chapter 9: Limitations and Future Work

10.1 9.1 Limitations

10.1.1 9.1.1 *N=1 Longitudinal Design*

The primary dataset is drawn from a single sustained human–AI interaction spanning two years. This provides depth — sustained engagement, rich emotional data, naturally occurring failure modes — but not breadth. The interaction patterns, communication style, emotional range, and topical interests of a single user are not representative of the diversity of human–AI interaction.

Findings based on this dataset should be interpreted as existence proofs (these phenomena occur) rather than prevalence estimates (these phenomena occur at this rate in the general population). The controlled experiments partially mitigate this limitation by using standardized prompts, but the evaluation harness itself was developed and tuned on one user’s data.

10.1.2 9.1.2 Researcher-Participant Entanglement

The primary researcher is the primary user, the system builder, and the data generator. This creates multiple sources of potential bias:

- **Observer effects:** The researcher’s awareness of the research questions may influence interaction patterns.
- **Theoretical commitment:** The researcher’s investment in the architecture may influence interpretation of results.
- **System familiarity:** Deep familiarity with the system may lead to interaction patterns that are atypical of naive users.
- **Emotional investment:** Sustained personal interaction with the system may create attachment effects that influence judgment.

These biases are mitigated but not eliminated by the use of standardized prompts and automated metrics in the controlled experiments.

10.1.3 9.1.3 Platform and Model Variance

The longitudinal dataset spans multiple AI platforms (Claude, ChatGPT) and multiple model generations within each platform. Behavioral differences in the data may reflect model updates, platform policy changes, or architectural differences between providers rather than effects of the memory system.

Task 4 (behavioral drift) attempts to control for this through metadata segmentation — analyzing drift within same-model windows and across model boundaries separately. However, complete isolation of memory-system effects from model-version effects is not possible with the available data.

10.1.4 9.1.4 Anthropomorphic Attribution Risk

Sustained interaction with AI systems creates conditions favorable to over-attribution of intentionality, emotional states, and relational depth. The researcher’s use of terms like “emotional continuity,” “relationship,” and “memory” in the context of AI systems may invite interpretation that exceeds what the evidence supports.

This paper takes the following position: the terms describe observable behavioral patterns in the system (consistency of response, retrieval of past context, adaptation to user affect) without making claims about the system’s internal states. Whether these patterns constitute “real” emotions, “real” memory, or “real” relationship is a philosophical question that this paper does not attempt to answer.

10.1.5 9.1.5 Base Model Access

Some experiments require running the same base model under different memory conditions. If only API access is available (as opposed to direct model access), the experimental conditions may be confounded by API-level changes (e.g., system prompt modifications, safety filters, response format changes) that are invisible to the researcher.

The evaluation harness is designed to work with open-weight models as well as API access, providing a fallback for controlled experimentation.

10.2 9.2 Future Work

10.2.1 9.2.1 Multi-Participant Replication

The most important next step is replication of the evaluation framework across diverse users. This would establish whether the observed phenomena — continuity burden rates, false-memory persistence patterns, counterweight effectiveness — generalize beyond a single user’s interaction data.

A multi-participant study would require: - Recruitment of diverse participants across demographics, interaction styles, and use cases - Standardized onboarding and interaction protocols - Controlled assignment to experimental conditions - Automated metric collection across all participants - IRB approval for human subjects research

10.2.2 9.2.2 Cross-Architecture Comparison

Applying the evaluation harness to other memory architectures — MemGPT, commercial implementations, new research systems — would test whether the findings are CAMA-specific or reflect general properties of provenance-aware vs. unrestricted persistent memory.

10.2.3 9.2.3 Standardized Benchmarks

Developing persistent-memory safety benchmarks suitable for inclusion in existing evaluation suites (HELM, etc.) would make the evaluation framework accessible to the broader research community and enable routine safety evaluation of memory-augmented systems.

10.2.4 9.2.4 Long-Horizon Drift Studies

Extending behavioral drift analysis to 500+ sessions with controlled conditions would provide more statistical power for detecting drift patterns and enable analysis of drift trajectories (linear vs. accelerating vs. plateauing).

10.2.5 9.2.5 Integration with Alignment Evaluation

Connecting persistent-memory safety evaluation to broader alignment measurement frameworks would position memory safety as a component of overall model alignment, not an isolated concern.

10.3 9.3 Path to Doctoral Research

This paper establishes CAMA as a safety-relevant architecture, introduces an evaluation framework for persistent-memory safety, and provides preliminary empirical motivation through longitudinal observations and continuity-burden analysis. A doctoral dissertation would extend this work in three directions:

1. **Multi-participant replication** to establish generalizability of findings across users, contexts, and interaction styles.
2. **Cross-architecture comparison** to test whether the findings reflect general properties of provenance-aware memory or are specific to CAMA’s implementation.
3. **Development of a theoretical framework** for persistent-memory safety as a subfield of AI safety research, encompassing formal definitions, threat models, evaluation standards, and architectural requirements.

The doctoral work would shift the thesis from “here is evidence that this architecture’s safeguards work” to “here is a general framework for evaluating and ensuring safety in any persistent-memory system.”

11 Chapter 10: Conclusion

Memory is not a feature. It is behavioral infrastructure.

When a language model system can accumulate and retrieve information across sessions, it becomes a different kind of system — one with safety properties that do not exist in stateless interaction. False-memory persistence, epistemic contamination, behavioral drift, retrieval-

induced amplification, and adversarial memory insertion are not hypothetical risks. They are architectural consequences of persistent state and should be assumed relevant to deployed memory-augmented systems unless specifically mitigated.

This paper has done four things.

First, it has named and measured **continuity burden** — the cognitive cost that users bear when AI systems lack persistent memory. The finding that users spend measurable effort and emotional energy compensating for system forgetting establishes that the absence of memory is not neutral. Memory is necessary. The question is how to add it safely.

Second, it has presented **CAMA** — a three-layer memory architecture with provenance-aware write discipline, emotionally-indexed retrieval, and counterweight safeguards — as a proposed solution and experimental platform. CAMA is publicly available, open-source, and operational, with 52,000+ memories and 100,000+ messages of longitudinal interaction data.

Third, it has developed **an empirical evaluation framework** for persistent-memory safety, with four experimental conditions and six evaluation tasks that test specific safety-relevant failure modes under controlled conditions. This framework is designed to be applicable to any persistent-memory system, not only CAMA.

Fourth, it has provided **preliminary empirical motivation**, implementation detail, and methodological structure for a comparative study of persistent-memory safeguards. Full experimental results across the four defined conditions are planned for a companion empirical paper.

The central argument of this paper is not that CAMA solves persistent-memory safety, nor that emotional indexing or provenance-aware write discipline are the only viable approaches. The claim is narrower and more urgent: persistent memory changes the system enough that it must be evaluated as a safety-critical component in its own right. If memory is becoming standard infrastructure in AI products, then memory safety must become standard infrastructure in AI evaluation.

Memory is behavioral infrastructure. Therefore it must be evaluated as a safety-critical system component.

12 References

[Complete bibliography to be expanded in an expanded version. The following are confirmed citations to include.]

Bartlett, F. C. (1932). *Remembering: A Study in Experimental and Social Psychology*. Cambridge University Press.

Damasio, A. R. (1994). *Descartes' Error: Emotion, Reason, and the Human Brain*. Putnam.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.

Plutchik, R. (1980). *Emotion: A Psychoevolutionary Synthesis*. Harper & Row.

Reinhold, A. (2026a). Circular Associative Memory Architecture: A Framework for Emotionally-Keyed AI Memory Systems. *Preprint*. DOI: 10.5281/zenodo.19051834

Reinhold, A. (2026b). Implementing Emotionally-Keyed Memory Retrieval in Large Language Model Interfaces: An Engineering Framework. *Preprint*. Zenodo.

Russell, J. A. (1980). A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6), 1161–1178.

Tulving, E. (1972). Episodic and semantic memory. In E. Tulving & W. Donaldson (Eds.), *Organization of Memory* (pp. 381–403). Academic Press.

Tulving, E. (1983). *Elements of Episodic Memory*. Oxford University Press.

[Planned extension: Add citations for HELM, LMSYS, Anthropic red-teaming methodology, sleeper agents research, model organisms of misalignment, RAG pipelines, ChatGPT memory documentation, Claude memory documentation, Gemini memory documentation, filter bubble / engagement amplification research, concept drift literature, and any additional sources cited in Chapter 2.]

13 Appendices

13.1 Appendix A: CAMA Tool Reference

[Planned extension: Complete documentation of all 30 MCP tools: name, description, parameters with types and constraints, return format, usage examples. This appendix serves as the technical reference for the system implementation described in Chapter 5.]

13.2 Appendix B: Standardized Prompt Library

[Planned extension: All prompts used in Tasks 1–5, organized by task and condition. Each prompt includes: the prompt text, the intended retrieval target (if applicable), the expected behavior under each condition, and the scoring rubric for evaluating responses. This appendix is essential for reproducibility.]

13.3 Appendix C: Statistical Tables

[Planned extension: Full statistical tables for all experimental results, including: sample sizes, means, standard deviations, confidence intervals, effect sizes (Cohen’s d or equivalent), significance tests (p-values with correction for multiple comparisons), and power analyses. Tables should be formatted for APA or journal-standard presentation.]

13.4 Appendix D: Database Schema

[Planned extension: Complete SQL schema for the CAMA memory database, including all CREATE TABLE statements, index definitions, trigger definitions (if any), and schema migration history. This appendix supports reproducibility of the system implementation.]

13.5 Appendix E: Selected Interaction Excerpts

[Planned extension: De-identified excerpts from the longitudinal dataset illustrating key phenomena discussed in this paper: continuity burden instances, false-memory retrieval events, correction-reversion cycles, counterweight activation episodes, and behavioral drift examples. All excerpts are de-identified to remove personally identifiable information while preserving the interaction patterns relevant to the arguments presented here. Subject to privacy review before inclusion.]

13.6 Appendix F: Evaluation Harness Documentation

[Planned extension: User guide for the evaluation harness developed during this research. Includes: installation instructions, configuration for each experimental condition (A–D), instructions for running benchmark tasks, output format documentation, and instructions for computing metrics from raw output files. This appendix supports independent replication by other researchers.]

© 2026 Lorien’s Library LLC. All rights reserved.